

Tutorial2: Advanced Programming Tasks of OCaml

1 练习题1：尾递归案例

```
let rec is_prime_helper x i =  
  if i == 1 then  
    true  
  else (x mod i != 0) && (is_prime_helper x (i-1))
```

- is_prime_helper 是尾递归，因为递归调用仅在 else 分支最后发生，因为 && 运算符求值顺序是从左到右。

2 练习题2：判断列表仅有一个元素

```
let has_single_elem l =  
  match l with  
  | [] -> false  
  | [_] -> true  
  | _ -> false;;  
(*或者*)  
let has_single_elem l =  
  match l with  
  | [] -> false;  
  | h :: t ->  
    if t = [] then true  
    else false;;  
has_single_elem [];; (* false *)  
has_single_elem [1];; (* true *)  
has_single_elem [2; 3; 4];; (* false *)
```

3 练习题3：列表最大数

```
let rec list_max l =  
  match l with  
  | [] -> failwith "Empty list."  
  | [x] -> x  
  | x :: xs -> max x (list_max xs)  
  
let rec list_max_iter l m =      (*tail recursive*)  
  match l with  
  | [] -> m  
  | x :: xs -> list_max_iter xs (max m x)
```

- list_max_iter无法还原 list_max 的 failwith

4 练习题4：用List.map实现列表元素平方

```
List.map (fun x -> x * x) [1;2;3;4;5];;
```

5 练习题5：重写List.filter函数

```
let rec filter func lst out=  
  match lst with  
  | [] -> out  
  | h :: t ->  
    if func h == true then filter func t (out @ [h])  
    else filter func t out;;  
  
filter (fun x -> x mod 2 = 0) [1; 2; 3; 4; 5; 6] [];
```

6 练习题6：列表中连续123的个数

```
let rec count_123 l =  
  match l with  
  | [] -> 0  
  | 1 :: 2 :: 3 :: xs -> 1 + (count_123 xs)  
  | x :: xs -> count_123 xs;;
```

7 练习题7：尾递归实现SUM函数

```
let rec sum_iter lst sum =  
  match lst with  
  | [] -> sum  
  | h :: t -> sum_iter t (sum + h);;  
  
sum_iter [2;3;4;5] 1;;
```

8 练习题8：差分序列求解

```
# let diff lin = (*tail recursive*)  
  let rec diff_in lin_in acc =  
    match lin_in with  
    | [] -> failwith "error!"  
    | [_] -> List.rev acc (* 结束条件：空列表或只有一个元素 *)  
    | h1 :: h2 :: t -> diff_in (h2 :: t) ((h2 - h1) :: acc) (* 递归调用 *)  
  in diff_in lin [] ;;  
val diff : int list -> int list = <fun>  
  
# diff [1;2;5;1] ;;  
- : int list = [1; 3; -4]
```

9 练习题9：fold_right实现

```
# let rec fold_right func lst num =  
  match lst with  
  | [] -> failwith "error"  
  | h :: t ->  
    if t == [] then func h num  
    else h - (fold_right func t num) ;;  
val fold_right : (int -> 'a -> int) -> int list -> 'a -> int = <fun>  
  
# fold_right (-) [1;3;3] 2 ;;  
- : int = -1
```

10 练习题10：自动机描述

- `smallbf [0;0;0] "++>>>+>+" = [4;0;1]`
- `smallbf [3;0;-1;-2] "+>+++++>>>>" = [4;5;-1;-2]`
- `smallbf [42] ">>>>>>>+>+>>" = [45]`

```
let pointer_next place max =  
  match place with  
  | x ->  
    if place < max - 1 then place + 1  
    else 0;;  
  
let rec nums_add nums place =  
  match nums with  
  | [] -> [] (* 如果列表为空, 返回空列表 *)  
  | h :: t ->  
    if place = 0 then  
      (h + 1) :: t (* 如果是第 place 位, 增加 1 *)  
    else  
      h :: nums_add t (place - 1) ;;  
  
let rec smallbfll nums str place =  
  match str with  
  | [] -> nums  
  | h :: t ->  
    if h == '+' then smallbfll (nums_add nums place) t place  
    else smallbfll nums t (pointer_next place (List.length nums));;  
  
let to_list str =  
  String.fold_right (fun x acc -> x :: acc) str [];;  
  
let smallbf nums str = smallbfll nums (to_list str) 0;;  
  
smallbf [0;0;0] "++>>>+>+";;
```

